

Privacy Preserving Collaborative IDS for Internet of vehicle networks

Click to edit Master subtitle style



Content:

1. High-Level Architecture (The Pipeline)
2. Pure Chebyshev Polynomial KAN (ChebyKAN) Trunk
3. Full Layer-wise Homomorphic Federated Learning (FHE)
4. Robust Outlier Aggregation (Multi-Krum on Ciphertexts)
5. Proof of Work & Federated Simulation Report

Click to edit Master subtitle style



Overall Architecture(phases):

Phase 1: Pure Chebyshev Polynomial KAN Model

- Input Normalization Layer mapping features to $[-1, 1]$
- Input Chebyshev KAN Linear Layer (46 features $\rightarrow$ 64 hidden nodes)
- Hidden Chebyshev KAN Linear Layers with LayerNorm and Dropout
- Output Chebyshev KAN Linear Layer mapping to class outputs

Phase 2: Layer-wise Homomorphic Federated Learning

- Dirichlet Non-IID Client Partitions ($\alpha = 0.3$)
- Full Layer-wise FHE Encryption (Each weight/bias tensor encrypted separately)



High-Level Architecture (The Pipeline)

Phase 2:

- Goal: Perform secure, privacy-preserving collaborative learning with ev
- • Step 1: Preprocess and scale datasets (percentile clipping, z-score)
- • Step 2: Dirichlet non-IID client sharding ($\alpha = 0.3$)
- • Step 3: Initialize Global Model (ChebyKAN) and register client vehicles
- • Step 4: Encrypt Global Model layers separately under FHE (Simulated
- • Step 5: Broadcast global layer-wise FHE ciphertexts to active clients
- • Step 6: Perform local client training and package layer-wise FHE client
- • Step 7: Homomorphic FedAvg + Multi-Krum aggregation layer-by-layer

Data Unification & Real-World Simulation

- Datasets Unified: CAN-VTC, Car-Hacking, CICIDS 2017, and VeReMi.
- • Stratified Splitting: 70% Train | 15% Validation | 15% Test.
- • Preprocessing stats (z-score, clipping) fitted strictly on training split to p
- Simulating the Fleet (Non-IID Distribution):
 - • Dirichlet allocation ($\alpha = 0.3$) shards training sets.
 - • Distributes data unevenly across simulated client nodes ($N = 10$ clients)
 - • Emulates class skew representing real-world Internet of Vehicles (IoV) e

Unified Dataset Adapter Interface:

```
def load_features_labels(root: Path, window: int, stride: int, max_rows:
Optional[int]) -> Tuple[np.ndarray, np.ndarray, Tuple[str, ...]]:
    feat_blocks, label_blocks = [], []
    for idx, cls in enumerate(CLASSES):
        # 1. Load the raw CAN-bus message attributes
        can_ids, dlc, payload = _parse_can_log(root / f"cleaned_{cls}_dataset.csv",
max_rows)

        # 2. Slice streams into sliding window features
        X = windows_from_stream(can_ids, dlc, payload, window, stride)
        y = np.full(X.shape[0], idx, dtype=np.int64)

        feat_blocks.append(X)
        label_blocks.append(y)

    # Returns unified shape: X: (N, 46), y: (N,)
    return np.concatenate(feat_blocks, axis=0), np.concatenate(label_blocks, axis=0),
CLASSES
```

Shows how raw data sources are read and converted into sliding windows of message streams.

For example, in `can_vtc_dataset.py` (and similarly for other logs), raw logs are parsed into numpy arrays, windowed, and labeled dynamically.



Unifying Heterogeneous Data to a Timeless 46-D Feature Vector

```
def extract_window_features(can_ids: np.ndarray, dlc: np.ndarray, payload:
np.ndarray) -> np.ndarray:
    """Unifies raw window logs into a timeless 46-dimensional feature vector."""
    feat = np.empty(46, dtype=np.float32)

    # 1. Payload byte distribution statistics (16 features)
    feat[0:8] = payload.mean(axis=0) # Mean of bytes 0..7
    feat[8:16] = payload.std(axis=0) # Std dev of bytes 0..7

    # 2. Length descriptor (2 features)
    feat[16] = dlc.mean()
    feat[17] = dlc.std()

    # 3. Frequency & Entropy descriptors (4 features)
    uniq, counts = np.unique(can_ids, return_counts=True)
    feat[18] = float(len(uniq)) / len(can_ids) # Unique CAN-ID ratio
    feat[19] = float(counts.max()) / len(can_ids) # Dominant ID frequency
    feat[45] = float((payload > 0).mean()) # Non-zero byte ratio

    # 4. Hashed CAN-ID Histogram distribution (24 features)
    buckets = np.zeros(24, dtype=np.float32)
    np.add.at(buckets, can_ids % 24, 1.0)
    feat[20:44] = buckets / buckets.sum()

    # 5. Shannon Entropy over payload bytes (1 feature)
    ent = 0.0
    for b in range(8):
        vals = (payload[:, b] * 255.0).astype(np.int32)
        _, c = np.unique(vals, return_counts=True)
        p = c.astype(np.float64) / c.sum()
        ent += float(-(p * np.log2(p + 1e-12)).sum())
    feat[44] = ent / 8.0

    return feat
```

Shows the mathematical/structural representation of the timeless CAN descriptors (from can_features.py) that enables dataset-agnostic training.

slots 0..7	payload byte means b0..b7	(normalised /255)
slots 8..15	payload byte stds b0..b7	
slots 16	DLC mean	
slots 17	DLC std	
slots 18	unique CAN-ID ratio	(count(unique IDs) / W)
slots 19	dominant-ID frequency	
slots 20..43	CAN-ID histogram over 24 hash buckets	
slots 44	mean Shannon entropy over the W payload byte distributions	
slots 45	non-zero payload byte ratio	

Total = 8 + 8 + 2 + 2 + 24 + 2 = 46 ✓

Normalization & Preprocessing

```
# 1. Stratified Partitioning
train_idx, val_idx, test_idx = _stratified_split(y, val_split, test_split, seed)

# 2. Percentile Clipping (fitted on training split only)
lo = np.quantile(X[train_idx], clip_low, axis=0)
hi = np.quantile(X[train_idx], clip_high, axis=0)
X = np.clip(X, lo, hi)

# 3. Z-score Standardisation (fitted on training split only)
mean = X[train_idx].mean(axis=0)
std = X[train_idx].std(axis=0) + 1e-6
X = (X - mean) / std

# 4. Outputs unified PyTorch datasets
return SplitResult(
    train=CanFeatureDataset(X[train_idx], y[train_idx]),
    val=CanFeatureDataset(X[val_idx], y[val_idx]),
    test=CanFeatureDataset(X[test_idx], y[test_idx]),
    ...
)
```

Shows how the splits are scaled identically to avoid data leakage (from build_dataset.py).



Dirichlet:

```
# Shard dataset indices across N clients using Dirichlet allocation
client_buckets = [[] for _ in range(n_clients)]

for c in classes:
    # 1. Get all sample indices for class 'c'
    idx_c = np.where(y == c)[0]
    rng.shuffle(idx_c)

    # 2. Draw proportions from Dirichlet distribution:  $p_c \sim \text{Dir}(\alpha * \mathbf{1}_N)$ 
    proportions = rng.dirichlet(alpha * np.ones(n_clients))

    # 3. Partition class 'c' indices based on these proportions
    splits = (np.cumsum(proportions) * len(idx_c)).astype(int)[: -1]
    chunks = np.split(idx_c, splits)

    # 4. Allocate chunks to corresponding clients
    for i, chunk in enumerate(chunks):
        if chunk.size:
            client_buckets[i].append(chunk)
```

It partitions a centralized dataset across N client vehicles such that each client receives a highly skewed, non-IID (non-independently and identically distributed) subset of data. This simulates realistic Internet of Vehicles (IoV) environments where different vehicles encounter different ratios of normal traffic and specific cyber-attacks.



Core Model & Cryptographic Security

- The Model: Pure Chebyshev KAN (ChebyKAN)
- • Chebyshev Polynomial KAN: Replaces splines with Chebyshev polynomials
- • Slices features directly into sequential Chebyshev basis projection layers
- Cryptographic Security (Layer-wise FHE):
 - • No Transport-Layer AES/ECDH: Handshakes, key exchanges, and symmetric encryption
 - • Full Layer-wise FHE Encryption: Clients encrypt each weight and bias tensor
 - • Homomorphic Aggregation: Aggregator server aggregates weights and

Chebyshev KAN & Adapting Weights

```
class KANConvNet(nn.Module):
    def __init__(
        self,
        in_features: int = 46,
        n_classes: int = 4,
        layer_size: int = 128,
        dropout: float = 0.2,
        fourier_modes: int = 8,
        conv_channels: Tuple[int, int] = (32, 64),
        kernel_size: int = 5,
        use_conv_backbone: bool = True,
    ) -> None:
        super().__init__()
        # 1. KAN convolutional backbone (Fourier-basis activations)
        if use_conv_backbone:
            c1, c2 = conv_channels
            self.conv1 = KANConv1d(in_features, c1, kernel_size,
                                   n_modes=fourier_modes)
            self.conv2 = KANConv1d(c1, c2, kernel_size, n_modes=fourier_modes)
            self.pool = nn.AdaptiveAvgPool1d(1)
            head_in = c2

        # 2. Fourier-basis Kolmogorov-Arnold Network Layer
        self.kan_in = FourierKANLinear(head_in, layer_size, n_modes=fourier_modes)
        self.act1 = KolmogorovActivation(layer_size)

        # 3. Hidden refinement (three Linear+ReLU blocks)
        self.hidden = nn.Sequential(...)
        self.act2 = KolmogorovActivation(layer_size)

        # 4. Final classification head
        self.head = nn.Linear(layer_size, n_classes)
```

Global Model Layout:

Features input scaling, recursive Chebyshev polynomial basis projection layers (T_0 to T_degree), stacked Chebyshev KAN Linear Layers with LayerNorm and Dropout, followed by a final output ChebyKAN layer mapping to classes.

Loading Weights with strict=False:

The server handles bootstrap weight adaptation to match the exact class structure of the target dataset, allowing the Chebyshev KAN model to support different output class dimensions.

```
# load pre-trained weights (e.g. from an intrusion detection baseline)
ckpt = torch.load(pretrained_path, map_location="cpu", weights_only=True)
sd = ckpt.get("model_state_dict", ckpt)

# Allow shape mismatches on the final classification head (strict=False)
# to automatically adapt the model to different dataset class counts
missing, unexpected = model.load_state_dict(sd, strict=False)
```

Chebyshev KAN Parameter Breakdown:

Total Parameters: 44,164 (CAN-VTC Configuration)

- Input Chebyshev KAN Layer (46 -> 64 hidden nodes):
 - Chebyshev weights: $64 * 46 * 5 = 14,720$ parameters
 - Base weights: $64 * 46 = 2,944$ parameters
 - Bias weights: 64 parameters
- Hidden Chebyshev KAN Layer (64 -> 64 hidden nodes):
 - Chebyshev weights: $64 * 64 * 5 = 20,480$ parameters
 - Base weights: $64 * 64 = 4,096$ parameters
 - Bias weights: 64 parameters
- Output Chebyshev KAN Layer (64 -> 4 class outputs):
 - Chebyshev weights: $4 * 64 * 5 = 1,280$ parameters
 - Base weights: $4 * 64 = 256$ parameters
 - Bias weights: 4 parameters



Layer-wise FHE Cryptography

CKKS Homomorphic Encryption Scheme:

Supports additions and scalar multiplications directly on real-numbered ciphertext vectors. The private key remains secret on the client (vehicle) and is never transmitted to the server.

```
@dataclass
class IdentityKey:
    name: str
    private: ed25519.Ed25519PrivateKey
    public: ed25519.Ed25519PublicKey

    @classmethod
    def generate(cls, name: str) -> "IdentityKey":
        sk = ed25519.Ed25519PrivateKey.generate()
        return cls(name=name, private=sk, public=sk.public_key())
```

Full Layer-wise FHE Processing:

Every weight and bias tensor is flattened, encrypted as a

```
# Sign the ephemeral exchange key to authenticate identity
signature = local_id.private.sign(eph_pub_bytes + peer_id_pub_bytes)

# Peer verifies signature with long-term public key to prevent MITM attacks
pending.peer_id_pub.verify(peer_signature, peer_eph_pub_bytes + local_id_pub_bytes)
```

ro



Homomorphic Aggregation & Outlier Filtering

Homomorphic FedAvg Aggregation:

Aggregates updates by evaluating additions and scalar weights homomorphically layer-by-layer:

$\text{Enc}(W_{\text{layer_global}}) = \text{sum}(w_i \cdot$

$\text{Enc}(W_{\text{layer_i}}))$

Homomorphic Multi-Krum Outlier Rejection:

Aggregator server computes Euclidean

distances between the encrypted weight updates (concatenating all layer ciphertexts)

to run scoring and filter out Byzantine clients before aggregation, protecting against model poisoning attacks.

End-to-End Privacy: No transport keys or AES; security is guaranteed by FHE mathematical bounds.

```
# 1. Generate fresh ephemeral ECDH-P256 keypair
eph = ec.generate_private_key(ec.SECP256R1())
eph_pub_bytes = eph.public_key().public_bytes(...)

# 2. Perform Elliptic Curve Diffie-Hellman (ECDH) key exchange
shared = pending.local_eph_priv.exchange(ec.ECDH(), peer_eph_pub)
```

```
# Deriving the 32-byte AES key using HKDF-SHA256
transcript = sorted_local_and_peer_ephemeral_pubkeys
aes_key = HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=transcript,
    info=b"FedIoV/v1/aes-256-gcm",
).derive(shared)
```



Optimization Setup (Genetic Algorithm)

- Purpose: Prepares the system for robust, privacy-preserving federated training across a vehicle fleet.
- • Simulation Grid Setup: Dirichlet concentration ($\alpha = 0.3$), sharding the dataset non-IID.
- • Client Settings: $N = 10$ simulated vehicle nodes, local epochs $E = 1$ per round, client sampling rate $q = 70\%$.
- • Optimizer Config: AdamW optimizer, batch size = 96, learning rate = $5e-4$.
- • Security Context: Simulated CKKS vector context representing FHE encryption.



Integrity Verification & Phase Boundaries

- Homomorphic Aggregation & Validation:
 - Server aggregates layer-wise FHE ciphertexts homomorphically.
 - Clients receive global ciphertexts and decrypt them using their private keys.
 - Integrity Check: Reconstructed weights match mathematical expectations.
- Complete FHE Simulation (Phase 2):
 - Centralized transport key handshakes (AES/ECDH) are fully deprecated.
 - Simulated federated rounds are executed sequentially across all 4 target devices.
 - Results compiled into a simulation report.

Proof of Work & Smoke Test Results

- System Sanity Check:
 - FHE Aggregation Time: Completed in ~3.5s to 18.8s depending on dataset
 - Outlier Rejection: Multi-Krum successfully filtered out Byzantine nodes
- Multi-Round Federated FHE Performance (3 Rounds):
 - CAN-VTC: 98.57% Accuracy | 0.9998 ROC-AUC
 - Car-Hacking: 60.43% Accuracy | 0.9360 ROC-AUC
 - CICIDS-2017: 99.58% Accuracy | 0.8236 ROC-AUC
 - VeReMi: 73.81% Accuracy | 0.5305 ROC-AUC (Reflects early round non-

